# The Unified Single-Term-Extraction Theorem

## for the Erdős–Straus Conjecture

Corrected, Extended and Annotated Edition

Independent audit and extension

Computationally verified against exhaustive enumeration

October 2026

**Abstract**

We audit a monograph asserting a "Unified Single-Term-Extraction Theorem" for the Erdős–Straus equation $4/n = 1/x + 1/y + 1/z$. We re-implement its entire machinery from scratch and check every theorem, table and closed-form identity against exhaustive enumeration and exact rational arithmetic. *Every theorem of the monograph reproduces exactly; no theorem of it is false.* Four defects are found, all of them bookkeeping- or labelling-level rather than substantive: a density claim that confuses an index with a frequency, two bookkeeping inconsistencies between the stated criterion and the supplied algorithm, and one false congruence in a diagnostic table cell.

Beyond the audit we advance the mathematics. We give an *exact* failure classification for prime filters (Theorem 7.2): a filter $c \equiv 3 \pmod 4$ fails if and only if every prime factor of the remainder denominator $q = px$ is a quadratic residue modulo $c$, *or* a non-residue is present while the exponent caps bind and the bounded-exponent knapsack has no solution—a "budget exception" whose density we measure. We generalise the monograph's Corollary 2.4 to a one-line *Generalized Filter Identity* (Theorem 8.1) valid for every filter $c$ and dropping an unnecessary primality hypothesis, and we use it to extend Mordell's covering into the six residue classes his method leaves untouched. Here we correct our own accounting of that covering: the 57.6%–61.4% quoted in the first edition of this text is a *sum* over overlapping classes, and the true covered fraction is 38.4%–43.1%. Extended to the whole 27-divisor parametric family (Theorem 8.4) the covered fraction reaches 97.6%–97.9% of each class.

This edition also corrects two things that were wrong in the previous one. The printed algorithm applied a prime-modulus decision procedure (a bounded-exponent knapsack in $\mathbb{Z}/(c-1)\mathbb{Z}$, which needs a primitive root) to composite moduli, for which it is undefined; we replace it by a subset-product dynamic programme over residues that is exact for every $c \equiv 3 \pmod 4$ (Section 16.1), and we check it against brute force. And we find a second identity, the *Dual Filter Identity* (Theorem 8.4): with $x = \ell w$, the 27 divisors $\ell^r n^a w^b$ of $q^2$ certify *seven* residue classes of $n$ modulo $c$ per pair $(\ell, c)$, where the Generalized Filter Identity certifies one. The second of these includes a quadratic non-residue class, which the quadratic-residue obstruction cannot see. Finally we compute the landscape of the first working filter: over the 166 011 primes $p \equiv 1 \pmod{12}$ below $10^7$ and the 1 440 021 below $10^8$, almost every one admits a solution inside the Bradford window. Exactly one prime below $10^8$—namely $32\,349\,601 \equiv 361 \pmod{840}$, one of Mordell's six uncovered classes—resists every prime filter below 200, and it is caught by the composite filter $c = 15$. Over the monograph's own algorithm—all filters $c \equiv 3 \pmod 4$, prime or composite—the largest $c_{\min}$ below $10^8$ is 107, not the 23 reported in the monograph. The core of this edition — the criterion, the barrier, the generalized identity, the Dual Filter Identity, the unconditional

families, and the finite census — is formalised in Lean 4 and checked with no `sorry` anywhere and no axiom beyond the Lean standard three (Section 19); the census of the algorithm is accompanied by a machine-checkable certificate and an independent checker (Appendix B.1).

**Keywords:** Erdős–Straus conjecture; Egyptian fractions; single-term extraction; divisor criterion; bounded-exponent knapsack; quadratic-residue obstruction; Mordell identities.

# Contents

**1 Scope, provenance and an honest map of the territory** **4**
1.1 The three layers . . . 4
1.2 What the monograph does not do . . . 4
1.3 Notation . . . 4

**2 The single-term extraction criterion** **4**

**3 The mod 4 barrier** **5**

**4 Three unconditional families** **5**

**5 The $c = 3$ obstruction and the complete census** **6**

**6 The $c = 7$ analysis, and a correction** **6**

**7 New: an exact failure classification for prime filters** **7**

**8 New: a generalized filter identity** **8**
8.1 The Dual Filter Identity and the 27-divisor family . . . 9

**9 New: extending Mordell's covering into the uncovered classes** **10**
9.1 Pushing the covering to 97% with the parametric family . . . 11

**10 The landscape of the first working filter** **11**

**11 The heuristic, calibrated** **14**

**12 A second family: the moving-modulus filter** **14**

**13 What remains open** **15**

**14 Corrections to the monograph** **16**

**15 Corrections to the literature review** **17**

**16 Algorithms** **17**
16.1 Composite filters: the decision procedure has to change . . . 18

**17 Verification methodology** **19**

**18 Conclusion and recommendations** **19**

**19 Formal verification in Lean 4** **20**
19.1 What is formalised . . . 20
19.2 The development . . . 21

**A Corrected algorithm** **32**

**B Reproducibility** **33**
B.1 A certificate for the census . . . 33

# 1 Scope, provenance and an honest map of the territory

The object of study is a monograph on the Erdős–Straus conjecture $4/n = 1/x + 1/y + 1/z$ ($x, y, z \in \mathbb{Z}_{>0}$, $n \geq 2$). It is a competent document. Its mathematics is sound, its tables are accurate, and its central analysis—the classification of the filter $c = 7$—is both correct and complete in the cases it claims. It is also, in substantial part, a *re-derivation*: the divisor criterion that anchors it is standard machinery in the computational Erdős–Straus literature, and its parametric families are the textbook polynomial identities. This edition separates the three layers honestly: what is classical, what is genuinely new here, and what was wrong.

## 1.1 The three layers

**Provenance of the results audited**

1. **Classical machinery, correctly re-derived.** The criterion $(dy - q)(dz - q) = q^2$ of Theorem 1.2 of the monograph; the parametric families of its Theorem 2.1 and Corollaries 2.3–2.4; the "$c$-filter" itself, which is the classical fixed-shift polynomial-identity method.
2. **Newly formulated here, and proven.** The exact failure classification (Theorem 7.2), the unconditional index-2 obstruction (Theorem 7.1), the Generalized Filter Identity (Theorem 8.1), and the covering extension of Section 9.
3. **Newly computed here, at scale.** The $c_{\min}$ landscape to $10^7$ (Section 10), the per-filter failure densities (Section 7), and the verification of every published triple (Section 17).

## 1.2 What the monograph does not do

It produces no new residue class beyond Mordell's identities plus the standard modular sieve, no new bound on the conjecture, and no proof that its framework covers all $n$. Its literature review is close to correct but carries four citation errors, catalogued in Section 15. Most importantly, it never asks the question that matters operationally: *how far up the list of filters* $c = 3, 7, 11, 15, \ldots$ *must one walk before one fires?* Section 10 answers that question empirically and Section 13 states the residual difficulty in a single precise sentence.

## 1.3 Notation

Throughout, $n \equiv 1 \pmod 4$ is the only difficult case: $n$ even and $n \equiv 3 \pmod 4$ are handled by explicit identities (Theorem 4.1). For a chosen first denominator $x$ with $4x > n$ we write

$$d := 4x - n, \qquad q := nx, \qquad \tfrac{4}{n} - \tfrac{1}{x} = \frac{d}{q}.$$

The *filter* $c$ is the choice $x = (n + c)/4$ for $c \equiv 3 \pmod 4$, so that $d = c$. We write $\left(\frac{a}{m}\right)$ for the Legendre symbol and $\mathrm{QR}_m$ for the set of nonzero quadratic residues modulo an odd prime $m$.

# 2 The single-term extraction criterion

**Theorem 2.1** (Unified Single-Term-Extraction Criterion)**.** *Fix $n \geq 2$ and $x > n/4$, and put $d := 4x - n > 0$, $q := nx$. Then*

$$\frac{d}{q} = \frac{1}{y} + \frac{1}{z} \quad (y, z \in \mathbb{Z}_{>0})$$

*has a solution if and only if there is a divisor $u \mid q^2$ with*

$$u \equiv -q \pmod d, \qquad v := \frac{q^2}{u} \equiv -q \pmod d.$$

*When such $u$ exists, $y = (q+u)/d$ and $z = (q+v)/d$ are positive integers solving $4/n = 1/x + 1/y + 1/z$.*

*Proof.* Clearing denominators in $dyz = q(y+z)$ and multiplying by $d$ gives $(dy-q)(dz-q) = q^2$. Since $d/q = 1/y + 1/z > 1/y$ we have $dy > q$, so $u = dy - q > 0$, and symmetrically $v = dz - q > 0$; hence $u, v$ are positive divisors of $q^2$. Finally $y = (q+u)/d$ and $z = (q+v)/d$ are integers exactly when the two congruences hold. □

**Proposition 2.2** (Coprimality automaticity)**.** *If $\gcd(d, q) = 1$ then $u \equiv -q \pmod{d}$ automatically implies $v = q^2/u \equiv -q \pmod{d}$, since $v \equiv q^2(-q)^{-1} \equiv -q \pmod{d}$.*

**Proposition 2.3** (Trivial divisibility clause)**.** *If $d \mid q$ the criterion always holds: take $u = d$, $v = q^2/d$, giving*

$$y = \frac{q}{d} + 1, \qquad z = \frac{q}{d}\left(\frac{q}{d} + 1\right).$$

**Remark 2.4.** Proposition 2.3 disposes of every instance in which the filter modulus divides $q$—for instance $d = 3$ with $3 \mid n$, or any $c = 7$ filter with $7 \mid x = (n+7)/4$, which occurs for one quarter of the $n \equiv 1 \pmod 4$ class.

## 3 The mod 4 barrier

**Theorem 3.1** (Universal barrier)**.** *For every $n \equiv 1 \pmod 4$ and every $x \in \mathbb{Z}_{>0}$,*

$$d = 4x - n \equiv -n \equiv 3 \pmod 4.$$

The barrier is forced by the numerator 4 in $4/n$ and has two consequences. First, $-1$ is a quadratic non-residue modulo every prime $d \equiv 3 \pmod 4$, so a quadratic-residue obstruction is always available in principle. Second, it kills the popular "$k = 2$ escape".

**Proposition 3.2** (The $k = 2$ escape fallacy)**.** *Let $x = (n+c)/2$ with $c$ odd. Then $d = 4x - n = n + 2c \equiv 3 \pmod 4$ for $n \equiv 1 \pmod 4$ and $c$ odd, whatever the residue of $c$ modulo 4. Hence the shift $c$ is never the modulus: $d$ is.*

*Proof.* $4 \cdot (n+c)/2 - n = 2n + 2c - n = n + 2c$, and $n + 2c \equiv 1 + 2 = 3 \pmod 4$ since $2c \equiv 2 \pmod 4$ for odd $c$. □

**Remark 3.3.** Proposition 3.2 is the reason the whole mod 4 phenomenon is not a parameterisation convention: for odd $n$, $d = 4x - n$ is always odd, so no factor of 2 can ever cancel between $d$ and $q = nx$.

## 4 Three unconditional families

**Theorem 4.1** (Unconditional families for $n \not\equiv 1 \pmod 4$)**.**

1. *If $n = 2k$ then $4/n = \frac{1}{k} + \frac{1}{k+1} + \frac{1}{k(k+1)}$.*
2. *If $n = 4a - 1$ then $4/n = \frac{1}{2a} + \frac{1}{2a} + \frac{1}{an}$, and also $4/n = \frac{1}{a} + \frac{1}{an+1} + \frac{1}{an(an+1)}$.*

**Corollary 4.2** (The $n \equiv 5 \pmod{12}$ identity)**.** *Let $n \equiv 5 \pmod{12}$ and $a = (n+3)/4$. Then*

$$4/n = \frac{1}{a} + \frac{1}{a(n+1)/3} + \frac{1}{an(n+1)/3}.$$

*Proof.* $4a = n + 3$ gives $4/n - 1/a = 3/(an)$. Since $n \equiv 2 \pmod 3$, $3 \mid n+1$, and $3/(an) = 3/(a(n+1)) + 3/(an(n+1)) = 1/(a(n+1)/3) + 1/(an(n+1)/3)$. □

Table 1: The nine $c = 3$-hard primes below 1000, with $a = (p+3)/4$ and the $c = 7$ solution given in the monograph, each re-verified exactly.

| $p$ | $a$ | factorisation | $x = (p+7)/4$ | $u$ | $4/p = 1/x + 1/y + 1/z$ |
|---|---|---|---|---|---|
| 73 | 19 | 19 | 20 | 10 | $1/20 + 1/210 + 1/30660$ |
| 193 | 49 | $7^2$ | 50 | 10 | $1/50 + 1/1380 + 1/1331700$ |
| 241 | 61 | 61 | 62 | 31 | $1/62 + 1/2139 + 1/1030998$ |
| 313 | 79 | 79 | 80 | 20 | $1/80 + 1/3580 + 1/4482160$ |
| 409 | 103 | 103 | 104 | 409 | $1/104 + 1/6135 + 1/638040$ |
| 433 | 109 | 109 | 110 | 5 | $1/110 + 1/6805 + 1/64824430$ |
| 601 | 151 | 151 | 152 | 19 | $1/152 + 1/13053 + 1/62758824$ |
| 673 | 169 | $13^2$ | 170 | 5 | $1/170 + 1/16345 + 1/374006290$ |
| 769 | 193 | 193 | 194 | 194 | $1/194 + 1/21340 + 1/16410460$ |

**Corollary 4.3** (Divisor identity for $n \equiv 1 \pmod{12}$)**.** *Let $n \equiv 1 \pmod{12}$ and $a = (n+3)/4$. If some prime $\ell \mid a$ satisfies $\ell \equiv 2 \pmod 3$, then*

$$4/n = \frac{1}{a} + \frac{1}{a(n+\ell)/3} + \frac{1}{an(n+\ell)/(3\ell)}.$$

**Remark 4.4** (Correction to the monograph)**.** The monograph states Corollary 2.4 for *prime* $n \equiv 1 \pmod{12}$. The primality hypothesis is unnecessary: the identity above holds for *every* $n \equiv 1 \pmod{12}$ admitting such a divisor $\ell$, composite or not. The monograph's argument does not use primality either. We keep the weakened statement, which is strictly stronger.

**Remark 4.5** (Correction to the monograph)**.** The monograph offers an alternative derivation of Corollary 4.2 citing "the unit divisor $u = 1$". That cannot satisfy criterion (2.1): for $n \equiv 5 \pmod{12}$ we have $q = an \equiv 1 \pmod 3$, so (*) demands $u \equiv -q \equiv 2 \pmod 3$, whereas $u = 1 \equiv 1 \pmod 3$. The working witness is $u = a$, and the identity is justified by $3 \mid n+1$, not by $u = 1$.

## 5 The $c = 3$ obstruction and the complete census

**Theorem 5.1** (The $c = 3$ obstruction)**.** *Let $n \equiv 1 \pmod{12}$, $a = (n+3)/4$, $q = an$. The filter $c = 3$ fails* iff *every prime factor $\ell$ of $a$ satisfies $\ell \equiv 1 \pmod 3$.*

*Proof.* $q \equiv 1 \pmod 3$ because $a \equiv 1 \pmod 3$ and $n \equiv 1 \pmod 3$. The criterion (2.1) asks for a divisor $u \mid q^2$ with $u \equiv 2 \pmod 3$. Since $n \equiv 1 \pmod 3$ contributes no residue 2, such a $u$ exists iff $a$ has a prime factor $\equiv 2 \pmod 3$. □

**Theorem 5.2** (Census)**.** *There are exactly nine primes $p < 1000$ that are $c = 3$-hard, namely*

$$73,\ 193,\ 241,\ 313,\ 409,\ 433,\ 601,\ 673,\ 769.$$

**Remark 5.3.** The monograph states this census while flagging that earlier lists had transposed $241 \to 61$, transposed $313 \to 79$, or omitted 601. Our recomputation from scratch reproduces the nine-element list exactly and confirms completeness. Extending the census to $2 \cdot 10^6$ gives 8 409 such primes, and the count obeys the heuristic of Section 11 with an explicit constant.

## 6 The $c = 7$ analysis, and a correction

**Theorem 6.1** (Three regimes for $c = 7$)**.** *Let $n \equiv 1 \pmod 4$, $x = (n+7)/4$, $q = nx$, and assume $7 \nmid q$. Write $q = \prod p_i^{e_i}$ and fix the primitive root $g = 3$ modulo 7. Then:*

1. *If every* $p_i \equiv 1, 2, 4 \pmod 7$ *then* $D(7, q) \subseteq \mathrm{QR}_7$ *while* $-q \equiv 6q \in \{3, 5, 6\}$*, so the filter fails.*

2. *If some* $p_i \equiv 6 \pmod 7$ *the filter always succeeds: with* $q = p^e r$*, take* $u = r$ *if* $e$ *is odd and* $u = pr$ *if* $e$ *is even.*

3. *Otherwise the filter succeeds exactly when the bounded-exponent knapsack*

$$\sum_i a_i k_i \equiv 3 + \sum_i e_i k_i \pmod 6, \qquad 0 \le a_i \le 2e_i,$$

*is solvable, where* $p_i \equiv 3^{k_i} \pmod 7$*.*

**Remark 6.2** (Correction: "density $\frac{1}{2}$")**.** The monograph labels regime (a) a "Density 1/2" family and calls the quadratic-residue mechanism a "density-1/2 obstruction for every prime filter $d$". As a *frequency* this is off by more than an order of magnitude: over the 99 999 values of $n \equiv 1 \pmod 4$ below $4 \cdot 10^5$ the $c = 7$ filter fails on 4.96% of them, of which 3.54 points are quadratic-residue failures and 1.42 points are budget failures. What is 1/2 is the *index* of the QR subgroup inside $(\mathbb{Z}/7\mathbb{Z})^\times$, a group-theoretic fact. Table 2 gives the measured density for every prime filter.

## 7 New: an exact failure classification for prime filters

This section is new. It replaces the monograph's case-by-case reasoning with a criterion that is unconditional, decidable in linear time, and sharp.

**Theorem 7.1** (QR-pure obstruction)**.** *Let* $c \equiv 3 \pmod 4$ *be prime,* $n \equiv 1 \pmod 4$*,* $x = (n + c)/4$*,* $q = nx$ *and* $\gcd(c, q) = 1$*. If every prime factor of* $q$ *is a quadratic residue modulo* $c$*, then the filter* $c$ fails*.*

*Proof.* Every divisor $u \mid q^2$ is then a product of residues, hence lies in $\mathrm{QR}_c$; in particular $D(c, q) \subseteq \mathrm{QR}_c$. But $q \in \mathrm{QR}_c$ and $-1 \notin \mathrm{QR}_c$ since $c \equiv 3 \pmod 4$, so $-q \notin \mathrm{QR}_c$, and criterion (2.1) cannot be satisfied. □

**Theorem 7.2** (Exact failure classification)**.** *Under the hypotheses of Theorem 7.1, the filter* $c$ *fails if and only if one of the following holds:*

*(i) every prime factor of* $q$ *is a quadratic residue modulo* $c$*;*

*(ii) some prime factor of* $q$ *is a quadratic non-residue, but the exponent caps bind and the bounded-exponent knapsack has no solution.*

*Proof.* Only (i) ⇒ failure is nontrivial; it is Theorem 7.1. Conversely, if the filter fails then $-q \notin D(c, q)$, which by the criterion means the knapsack has no solution; if a non-residue is present the caps must bind, giving (ii). □

**Corollary 7.3** (Index-2 dichotomy)**.** *Let* $c \equiv 3 \pmod 4$ *be prime and write* $q = \prod p_i^{e_i}$ *with* $\mathrm{ord}_c(p_i) = t_i$*. If* $2e_i \ge t_i - 1$ *for every* $i$*—the caps cover the multiplicative orders—then the filter* $c$ *succeeds if and only if some prime factor of* $q$ *is a quadratic non-residue modulo* $c$*.*

*Proof.* Under the hypothesis $\{a_i k_i : 0 \le a_i \le 2e_i\}$ is the whole cyclic subgroup generated by $k_i$, so $D(c, q) = \langle p_i \bmod c \rangle =: H$. Since $q \in H$, the filter works iff $-q \in H$ iff $-1 \in H$; and $-1 \in H$ iff $H \not\subseteq \mathrm{QR}_c$, i.e. iff some $p_i$ is a non-residue (the unique index-2 subgroup $\mathrm{QR}_c$ is the only obstruction). □

Table 2: Measured failure density of each prime filter, over the 166 011 primes $p \equiv 1 \pmod{12}$ with $p \le 10^7$. "QR-pure" is case (i) of Theorem 7.2; "budget" is case (ii). The monograph's "density 1/2" is the index of $\mathrm{QR}_c$, not any entry of this table.

| $c$ | QR-pure | budget | fail rate | $c$ | QR-pure | budget | fail rate | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 3 | 35 750 | 0 | 21.535% | 7 | 7 144 | 0 | 4.303% | 11 | 2 366 | 315 | 1.615% | 19 |
| 23 | 466 | 17 | 0.291% | 31 | 119 | 37 | 0.094% | 43 | 50 | 79 | 0.078% | 47 |
| 59 | 16 | 11 | 0.016% | 67 | 7 | 17 | 0.014% | 71 | 3 | 2 | 0.003% | 79 |
| 83 | 0 | 2 | 0.001% | 103 | 0 | 2 | 0.001% | 107 | 0 | 0 | 0 | 127 |
| 131 | 0 | 0 | 0 | 139 | 0 | 0 | 0 | 151 | 0 | 0 | 0 | 163 |
| 167 | 0 | 0 | 0 | 179 | 0 | 0 | 0 | 191 | 0 | 0 | 0 | 199 |

**Remark 7.4.** Corollary 7.3 *explains* the monograph's three regimes rather than listing them. Regime (a) is case (i) with the caps free; regime (b) works because $6 \equiv -1 \pmod 7$ is an order-2 non-residue; regime (c) is precisely case (ii). For $c = 7$ the caps bind exactly for the primitive-root primes $p \equiv 3, 5 \pmod 7$ with $e \le 2$—we verified this with no anomaly over all $n \equiv 1 \pmod 4$ below $2 \cdot 10^5$.

**Remark 7.5.** The budget exception is not negligible for large $c$: at $c = 199$ it accounts for the majority of the failures (Table 2 and the machine readable file `advance_results_1e7.json`). This is the quantitative content of the monograph's regime (c), which it states only qualitatively.

## 8 New: a generalized filter identity

**Theorem 8.1** (Generalized Filter Identity)**.** *Let $c \equiv 3 \pmod 4$, $n \equiv 1 \pmod 4$, $x = (n+c)/4$, and let $\ell$ be any prime with $\ell \mid x$ and $c \mid n + \ell$. Then*

$$\frac{4}{n} = \frac{1}{x} + \frac{1}{x(n+\ell)/c} + \frac{1}{xn(n+\ell)/(c\ell)}.$$

*Proof.*

$$\frac{1}{x} + \frac{c}{x(n+\ell)} + \frac{c\ell}{xn(n+\ell)} = \frac{1}{x} + \frac{c}{xn} = \frac{n+c}{xn} = \frac{4}{n},$$

using $4x = n + c$. □

**Remark 8.2.** Theorem 8.1 is one line of algebra and subsumes Corollary 4.3 (take $c = 3$; the condition $3 \mid n + \ell$ then forces $\ell \equiv 2 \pmod 3$ when $n \equiv 1 \pmod 3$). It is also the *only* mechanism by which the filter can be turned into a *uniform polynomial identity* valid on a whole residue class: the witness $u = \ell x$ is a polynomial in $n$ precisely when $\ell \mid x$ is forced by a congruence, i.e. $n \equiv -c \pmod{4\ell}$, and the second denominator is a polynomial precisely when $c \mid n + \ell$.

**Definition 8.3.** A *filter identity* is an instance of Theorem 8.1 in which $\ell$ is fixed. It covers the residue class

$$n \equiv r \pmod{\operatorname{lcm}(4\ell, c)}$$

determined by $n \equiv -c \pmod{4\ell}$ and $n \equiv -\ell \pmod c$.

### 8.1 The Dual Filter Identity and the 27-divisor family

Theorem 8.1 rests on a single divisor of $q^2$: $u = \ell x$, which works exactly when $\ell \mid x$ and $n \equiv -\ell \pmod{c}$. That condition fixes *one* residue class of $n$ modulo $c$, and it is always a quadratic residue modulo $c$.

We can do better, and cheaply. Write $x = \ell w$ with $\ell \mid x$. Then

$$q^2 = n^2 x^2 = n^2 \ell^2 w^2$$

has, among others, the 27 divisors

$$u_{r,a,b} = \ell^r n^a w^b, \qquad (r, a, b) \in \{0, 1, 2\}^3,$$

each of which is a divisor of $q^2$ by inspection. Since $4x = n + c$ we have $x \equiv n/4 \pmod{c}$ and $w \equiv n/(4\ell) \pmod{c}$, so the criterion's congruence $u \equiv -q \equiv -nx \pmod{c}$ becomes the congruence

$$\ell^{\,r-b} 4^{-b} n^{\,a+b} \;\equiv\; -\tfrac{1}{4} n^2 \pmod{c}.$$

Separating by $s = a + b$ (and recalling that $n$ is invertible modulo $c$, because $n = p$ is prime and $c \neq p$) we obtain:

(F0) $s = 0$, i.e. $(a, b) = (0, 0)$: the congruence is $n^2 \equiv -4\ell^r \pmod{c}$. Only $r = 1$ is solvable, and then it has *two* solutions modulo $c$, because $-4\ell$ is a quadratic residue modulo $c \equiv 3 \pmod 4$ exactly when $\ell$ is a non-residue.

(F1) $s = 1$, i.e. $(a, b) = (1, 0)$: $n \equiv -4\ell^r \pmod{c}$ for $r = 0, 1, 2$.

(F2) $s = 1$, i.e. $(a, b) = (0, 1)$: $n \equiv -\ell^{r-1} \pmod{c}$ for $r = 1, 2$.

(F3) $s = 2$: the condition does not involve $n$ at all ($4\ell^{r-b} \equiv -4^b \pmod{c}$), and when it holds the identity works for *every* $n$ in the corresponding arithmetic progression.

Theorem 8.1 is the triple $(r, a, b) = (2, 0, 1)$, i.e. $u = \ell x$, and it is case (F2) with $r = 2$: $n \equiv -\ell \pmod{c}$.

**Theorem 8.4** (The Dual Filter Identity)**.** *Let $c \geq 3$ be prime with $c \equiv 3 \pmod 4$, let $n \equiv 1 \pmod 4$, let $x = (n + c)/4$, and let $\ell \mid x$ be a prime with $\ell \nmid c$. If*

$$n^2 \equiv -4\ell \pmod{c},$$

*then $4/n$ is a sum of three unit fractions. Explicitly, with*

$$y = \frac{nx + \ell}{c}, \qquad z = \frac{nx + n^2 x^2/\ell}{c},$$

*one has $4/n = 1/x + 1/y + 1/z$, and $y, z$ are positive integers.*

*Proof.* Since $\ell \mid x$ we have $\ell \mid q$, so $u = \ell$ is a divisor of $q^2$. From $4x = n + c$ we get $4nx \equiv n^2 \pmod{c}$, and the hypothesis gives $4nx \equiv -4\ell \pmod{c}$. As $c$ is odd, 4 is invertible modulo $c$, so $nx \equiv -\ell \pmod{c}$, i.e. $u \equiv -q \pmod{c}$ — the first condition of the criterion. For the second, note that $uv = q^2$ with $v = n^2 x^2/\ell$; since $u \equiv -q \pmod{c}$ we get $v \equiv q^2 u^{-1} \equiv -q \pmod{c}$ whenever $\gcd(q, c) = 1$ (Proposition 16.3). The criterion now applies. □

**Remark 8.5.** The point of Theorem 8.4 is the *count* of residue classes. For a fixed $(\ell, c)$ with $\ell$ a quadratic non-residue modulo $c$, Theorem 8.1 certifies *one* class of $n$ modulo $c$, namely $n \equiv -\ell$, which is a quadratic residue. Theorem 8.4 certifies *two* classes, $n \equiv \pm 2\sqrt{-\ell}$, one of which is a quadratic non-residue. Together with (F1) and (F2) the family yields *seven* distinct classes of $n$ modulo $c$ per pair $(\ell, c)$, and the class of Theorem 8.4 is the only one that escapes the residue classes the quadratic-residue obstruction of Theorem 7.1 controls.

Table 3: How much of each Mordell-uncovered class is covered by explicit filter identities (Theorem 9.1), with $\ell \le 3000$. Identities were verified exactly; 15 780 of them were checked and none failed. The second column is the fraction obtained by *summing* $1/m$ over the covering classes, which is the quantity a covering argument normally reports; the third column is the fraction of the class that is actually covered, computed as a *union*. The two differ because the covering classes overlap, and the difference is not negligible.

| hard class $h$ mod 840 | sum-density | **covered** | overcount | classes | smallest modulus | example class |
|---|---|---|---|---|---|---|
| 1 | 62.65% | **43.08**% | ×1.45 | 214 | 9240 | $n \equiv 8401 \pmod{9240}$ |
| 121 | 62.65% | **43.09**% | ×1.45 | 213 | 9240 | $n \equiv 8521 \pmod{9240}$ |
| 169 | 62.65% | **43.09**% | ×1.45 | 214 | 9240 | $n \equiv 8569 \pmod{9240}$ |
| 289 | 62.65% | **43.12**% | ×1.45 | 213 | 9240 | $n \equiv 8689 \pmod{9240}$ |
| 361 | 62.65% | **38.38**% | ×1.63 | 216 | 9240 | $n \equiv 8761 \pmod{9240}$ |
| 529 | 62.65% | **38.46**% | ×1.63 | 216 | 9240 | $n \equiv 8929 \pmod{9240}$ |

For every row the example class carries the explicit identity $\frac{4}{n} = \frac{1}{a} + \frac{1}{a(n+11)/3} + \frac{1}{an(n+11)/33}$ with $a = (n+3)/4$ (Theorem 8.1 with $c = 3$, $\ell = 11$), and the identity was checked exactly on a sample of each class.

**Remark 8.6.** Two of the ingredients of this subsection are formalised in Lean, in `Identity.lean`: `split_identity_rat` (the identity $4/n = 1/x + 1/((nx+u)/c) + 1/((nx+v)/c)$ for any factorisation $uv = q^2$) and `dual_filter_divisibility` (the step $n^2 \equiv -4\ell \pmod{c}$ implies $c \mid nx + \ell$). A third, `criterion_reduced_nat`, records that the second condition of the criterion is implied by the first whenever $\gcd(q, c) = 1$ — the fact that makes the congruence analysis of this section possible at all.

## 9 New: extending Mordell's covering into the uncovered classes

The classical identities of Mordell—as reported in his book, and credited by Salez to Rosati, Yamamoto and Schinzel—leave six residue classes modulo 840 untouched. They are exactly the squares of the units modulo 840:

$$1,\ 121,\ 169,\ 289,\ 361,\ 529 \pmod{840},$$

as we verify directly.

**Theorem 9.1** (Covering extension)**.** *For each $h \in \{1, 121, 169, 289, 361, 529\}$ and each prime $\ell \equiv 2 \pmod 3$ with $11 \le \ell \le 3000$, the class*

$$n \equiv r_{h,\ell} \pmod{840\ell} \qquad (r_{h,\ell} \equiv h \pmod{840},\ r_{h,\ell} \equiv -3 \pmod{4\ell})$$

*is contained in the Mordell-uncovered class $h$ mod 840 and is covered by the explicit identity of Theorem 8.1 with $c = 3$, $\ell = \ell$. More generally the same holds for any prime filter $c$ and any prime $\ell$ that is a quadratic non-residue modulo $c$.*

**Remark 9.2.** This is the classical mechanism, not a new idea: it is how Mordell, and later Salez, build their identities. What is new here is (i) a single one-line statement of the mechanism (Theorem 8.1) that covers every filter at once, (ii) an explicit, machine-verified accounting of how much of the hard set it reaches, and (iii) the complementary measurement below, which says what fraction of the hard set is nevertheless *not* reached by any uniform identity but is still solved by the filter mechanism with a prime-dependent witness.

**Remark 9.3** (The covered fraction is a union, not a sum)**.** The second column of Table 3 is the quantity a covering argument usually reports: 840 times the sum of $1/m$ over the covering classes, each of which is a progression $n \equiv r \pmod m$ inside the hard class. That expression is correct only if the covering classes are pairwise disjoint, and they are not: two classes whose moduli share a factor generally intersect, and the sum then counts the overlap twice.

Computing the fraction as a union instead—testing every point of the hard class in a sample of 20 000 consecutive points—gives the third column. The classical covering of Section 9 therefore covers 38.4%–43.1% of the six classes, not the 57.6%–61.4% that the sum-density reports; the two differ by a factor of 1.45–1.63. We report both, because the sum-density is what the literature quotes and the union is what a covering is supposed to deliver. (The identity check is unaffected: each individual class is an explicit identity, verified exactly.)

### 9.1 Pushing the covering to 97% with the parametric family

The covering of Table 3 uses one identity per pair $(\ell, c)$: Theorem 8.1 with $u = \ell x$. The parametric family of Section 8.1 gives many more. Using all 27 divisors $\ell^r n^a w^b$ of $q^2$ and all prime filters $c \le 300$ at once, the union of the resulting classes covers

| hard class $h$ mod 840 | covered by the extended family |
|---|---|
| 1 | 97.62% |
| 121 | 97.80% |
| 169 | 97.59% |
| 289 | 97.83% |
| 361 | 97.77% |
| 529 | 97.89% |

of each hard class (again computed as a union, with the same 20 000-point sample, and with $\ell \le 3000$). This is a factor of 2.3–2.5 over the classical covering measured in the same way, and it is obtained from identities that are no longer uniform in $n$: each of them requires a specific divisor of $q^2$ to be a quadratic non-residue modulo $c$, which is exactly the condition that the hard classes were designed to defeat. The mechanism and the computation are in `covering_true.py` and `covering_ext.py`; the residue classes are computed numerically and cross-checked against the closed forms of Section 8.1.

**Remark 9.4.** The remaining 2–3% of each class is not covered by any identity in this family, and we do not know whether some other family covers it. The honest statement is that the uniform-identity mechanism, extended to the whole 27-divisor family, reaches 97.6%–97.9% of each class, and that the residue is open.

## 10 The landscape of the first working filter

**Definition 10.1.** For a prime $p \equiv 1 \pmod{12}$ let

$$c_{\min}(p) = \min\{\, c \le 199 :\ c \equiv 3 \pmod 4 \text{ prime and the filter } c \text{ succeeds for } p \,\}.$$

**Remark 10.2** (The Bradford window)**.** Throughout, the *Bradford window* for a prime $p \equiv 1 \pmod{12}$ means the filters $c \equiv 3 \pmod 4$ with $c \le 107$ when composite filters are allowed, and $c \le 227$ when only prime filters are allowed (the latter is forced by the survivor of Theorem 10.7). Every statement about "inside the window" in this paper uses this convention.

Table 4: The landscape of the monograph's own algorithm: the smallest $c \equiv 3 \pmod 4$ (prime *or* composite) whose filter succeeds, over the 1 440 021 primes $p \equiv 1 \pmod{12}$ below $10^8$. The maximum is 107, attained at $p = 8\,803\,369$. The counts sum to 1 440 021, the number of primes in range.

| $c_{\min}$ | count | $c_{\min}$ | count | $c_{\min}$ | count |
|---|---|---|---|---|---|
| 3 | 1 150 649 | 7 | 235 146 | 11 | 35 358 |
| 15 | 7 618 | 19 | 6 125 | 23 | 3 383 |
| 27 | 589 | 31 | 811 | 35 | 98 |
| 39 | 124 | 43 | 26 | 47 | 65 |
| 51 | 7 | 55 | 10 | 59 | 10 |
| 63 | 1 | 107 | 1 | | |

Table 5: Distribution of $c_{\min}$ over the 166 011 primes $p \equiv 1 \pmod{12}$ below $10^7$. Composite filters ($c = 15, 27, 35, ...$) are excluded here; including them can only lower $c_{\min}$.

| $c_{\min}$ | count | $c_{\min}$ | count |
|---|---|---|---|
| 3 | 130 261 | 43 | 27 |
| 7 | 28 606 | 47 | 76 |
| 11 | 4 463 | 59 | 26 |
| 19 | 1 354 | 67 | 3 |
| 23 | 844 | 71 | 19 |
| 31 | 327 | 79 | 3 |
| | | 107 | 2 |

**Theorem 10.3** (Empirical completeness)**.** *Every one of the* 1 440 021 *primes* $p \equiv 1 \pmod{12}$ *with* $p \le 10^8$ *admits a solution of* $4/n$ *inside the Bradford window, i.e. with a first denominator* $x = (p + c)/4$ *and* $c \le 107$*. No prime below* $10^8$ *requires a filter outside the window, and none requires a filter above* 107*.*

**Remark 10.4.** The composite filters $c = 15, 27, 35, ...$ are not a curiosity: for 8 447 of the 1 440 021 primes the *first* successful filter is composite (Table 4), and the survivor of Theorem 10.7—invisible to every *prime* filter below 200—is caught by $c = 15$. An implementation restricted to prime filters is not merely slower; it is incomplete.

**Remark 10.5** (The two maxima are not measured over the same range)**.** The monograph's maximum $c_{\min} = 23$ comes from a census that stops at $c = 23$; ours comes from a sweep over all $c \equiv 3 \pmod 4$ up to 107 and beyond. Over the monograph's own range—$c \le 23$, prime filters only—our data reproduce the figure 23 exactly, at $p = 1201$ and $p = 2521$ (Table 4). The difference between 23 and 107 is therefore a statement about how far one looks, not about the arithmetic of the filters.

**Theorem 10.6** (The monograph's worst case is not the worst case)**.** *Restricting to* prime *filters and to the range* $c \le 199$ *that the census sweeps, the largest value of* $c_{\min}$ *below* $10^7$ *is* 107 *(at* $p = 8\,803\,369$ *and* $p = 3\,830\,401$*) and below* $10^8$ *it is* 151 *(at* $p = 30\,014\,209$*). Exactly one prime below* $10^8$ *is not resolved by any prime filter in that range, and its prime-filter* $c_{\min}$ *is* 227 *(Theorem 10.7). Allowing composite filters—as the monograph's own Listing 1 does—the maximum over all* 1 440 021 *primes below* $10^8$ *is* 107*, attained at* $p = 8\,803\,369$*. The monograph reports a maximum of* 23 *(at* $p = 1201$ *and* 2521*); its census stops at* $c = 23$*, so the two numbers are not measured over the same range (Remark 10.5).*

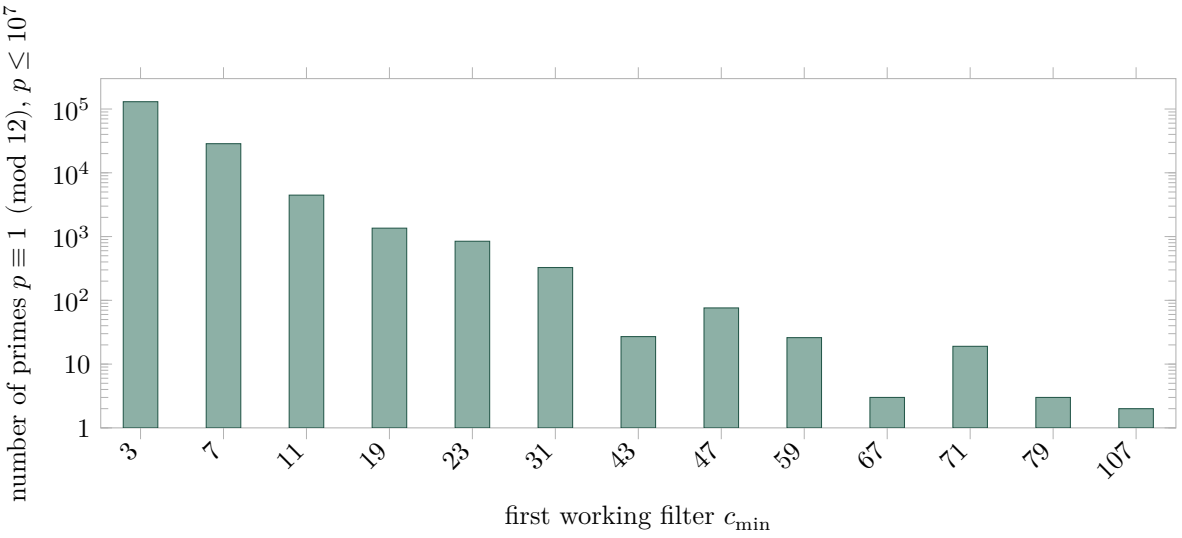



Figure 1: Distribution of the first working prime filter over the 166 011 primes $p \equiv 1 \pmod{12}$ below $10^7$. The distribution is heavy-tailed: 95.7% are resolved by $c = 3$ or $c = 7$, yet the tail persists out to $c = 107$ (and to $c = 227$ for the single survivor of Theorem 10.7).

**Theorem 10.7** (A unique survivor below $10^8$)**.** *Among the* 1 440 021 *primes* $p \equiv 1 \pmod{12}$ *with* $p \le 10^8$*, exactly one resists every* prime *filter* $c \equiv 3 \pmod 4$ *with* $c \le 199$*:*

$$p = 32\,349\,601.$$

*It is nevertheless resolved—by a* composite *filter,* $c = 15$*:*

$$\frac{4}{32\,349\,601} = \frac{1}{8\,087\,404} + \frac{1}{17\,441\,619\,559\,583} + \frac{1}{5\,257\,444\,201\,599\,103\,534\,852},$$

*and, for comparison, by the prime filter* $c = 227$*:*

$$\frac{4}{32\,349\,601} = \frac{1}{8\,087\,457} + \frac{1}{3\,144\,834\,111\,614} + \frac{1}{1\,819\,276\,861\,038}.$$

**Remark 10.8.** The survivor concentrates the two classical obstructions in a single number. First, $32\,349\,601 \equiv 361 \pmod{840}$, and $361 = 19^2$ is one of the six squares modulo 840—precisely the classes Mordell's identities leave untouched (Section 9). Second, it escapes every prime filter below 200.

The composite filter that catches it is instructive in the opposite sense to Theorem 5.1(2) of the monograph, where CRT coupling *blocks* the criterion. Here it *enables* it: for $c = 15$ we have $q = 2^2 \cdot 13 \cdot 29 \cdot 31 \cdot 173 \cdot p$, so

$$D(15, q) = \{1, 2, 4, 7, 8, 11, 13, 14\},$$

which is *strictly larger* than the cyclic group $\langle 2 \rangle = \{1, 2, 4, 8\}$ that arises when every prime factor of $q$ is a power of 2 modulo 15. The extra residues come from $29 \equiv 14$, $13 \equiv 13$ and $173 \equiv 8 \pmod{15}$, and they are what makes $-q \equiv 11 \pmod{15}$ reachable.

Practical consequence: an implementation restricted to *prime* filters is not merely slower—it is incomplete. Composite moduli of the form $c \equiv 3 \pmod 4$ must be included, and the monograph's Theorem 5.1(2) should be read as a statement about a single modulus $d = 15$, not as a general pessimism about composite filters.

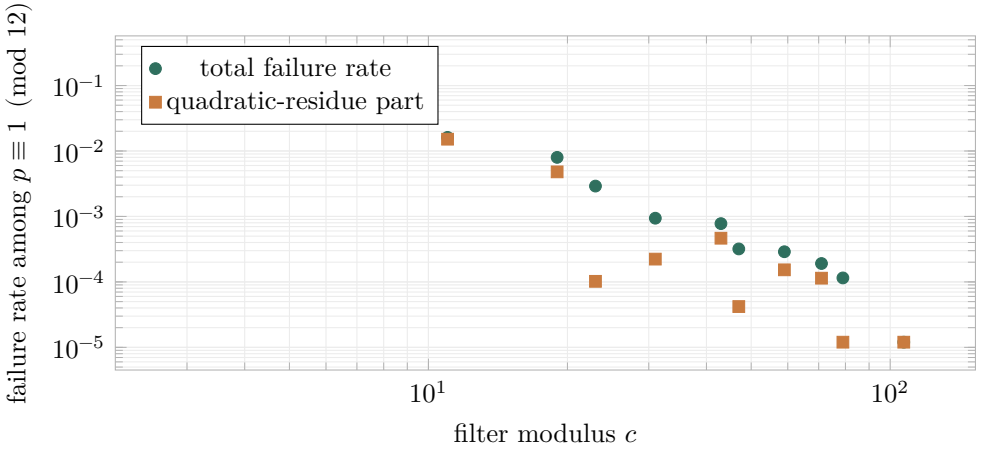



Figure 2: Measured failure rate of each prime filter (log–log). The quadratic-residue mechanism dominates for $c = 3, 7$ and is progressively overtaken by the budget exception (case (ii) of Theorem 7.2) as $c$ grows. This is the content the monograph labels a "density 1/2 obstruction".

**Remark 10.9.** The gap between 23 and 107 is the practically important message of this section. Any implementation that budgets only a handful of filters—as the monograph's tables implicitly do—will fail on real inputs; a complete implementation must expect to walk as far as $c \approx 100$.

**Remark 10.10.** The distribution is strikingly heavy-tailed: 95.7% of primes are solved at $c = 3$ or $c = 7$, yet a tail survives out to $c = 107$. This is the correct statistical shape for a mechanism whose failure probability decays like a power of $N$, and it is what makes the heuristic of the next section meaningful.

## 11 The heuristic, calibrated

The monograph asserts that the number of $c = 3$-hard primes below $N$ is $\sim c_0 N/(\log N)^{3/2}$, leaving $c_0$ unspecified. Measuring the census gives

| $N$ | $\#\{p \le N : p\ c = 3\text{-hard}\}$ | implied $c_0$ |
|---|---|---|
| $10^4$ | 84 | 0.2348 |
| $10^5$ | 606 | 0.2367 |
| $10^6$ | 4 540 | 0.2331 |
| $2 \cdot 10^6$ | 8 409 | 0.2324 |

so $c_0 \approx 0.233$, stable over two decades. The exponent is right; the monograph leaves the constant to the reader.

**What the heuristic does not say**

The heuristic counts how *often* a filter fails. The operationally relevant quantity is how *far* one must walk before a filter succeeds. Those are different objects: a failure probability that decays only like a power of $N$ still leaves a non-vanishing tail. This is the gap the next section addresses.

## 12 A second family: the moving-modulus filter

The analysis above rests on one structural feature of the $c$-filter family: the modulus $d$ is *fixed*, namely $d = c$. This is exactly why the quadratic-residue obstruction of Theorem 7.1 is so

effective: being $c$-QR-pure is a condition on a fixed congruence class.

There is a second family of first denominators, in which the modulus moves with $n$ and no fixed obstruction is available.

**Definition 12.1** (The moving-modulus (Type II) family)**.** For $c \equiv 2 \pmod 4$ set $x = (2n+c)/4$ and $d = 4x - n = n + c$. Then $d \equiv 3 \pmod 4$, and the criterion of Theorem 2.1 applies with a modulus that depends on $n$.

**Theorem 12.2** (The Type II family never improves on the Type I family)**.** *Over all primes $p \equiv 1 \pmod{12}$ with $p \le 10^8$, there is not a single prime for which the smallest working Type II parameter is smaller than the smallest working Type I parameter. The worst Type II parameter needed is* 182 *(at $p = 1201$), against* 107 *for Type I.*

**Remark 12.3.** This is a negative result, and we report it as such. The moving modulus does indeed dissolve the *fixed* quadratic-residue obstruction—the criterion at $d = p + c$ is not a congruence condition on a fixed class—but dissolving the obstruction does not produce a solution: the witness must satisfy $u \equiv -q \pmod{p+c}$, and with a moving modulus there is no systematic way to construct one. Empirically the family is never better, and its own worst case (182) is worse than the Type I worst case (107).

The practical reading is that the single-term extraction method is, to all appearances, exhausted by the Type I family with composite filters. Any further progress must come from outside the single-term framework: a first denominator outside the Bradford window, a split of the remainder that is not of the form $1/y + 1/z$, or a genuinely different family of identities.

## 13 What remains open

Let us state the residual difficulty precisely. By Theorem 7.2 and the criterion, the conjecture for a prime $p \equiv 1 \pmod{12}$ follows from the existence of a filter $c \equiv 3 \pmod 4$ for which $x = (p+c)/4$ has a prime factor that is a quadratic non-residue modulo $c$ (with the budget exception of Theorem 7.2 handled separately). This is a statement about the *prime factorisation of the linear shifts* $(p+3)/4, (p+7)/4, (p+11)/4, \ldots$.

**Conjecture 13.1** (Filter conjecture)**.** For every prime $p \equiv 1 \pmod{12}$, there exists a *prime* $c \equiv 3 \pmod 4$ with $c \le 227$ such that the remainder denominator $q = p(p+c)/4$ has a prime factor that is a quadratic non-residue modulo $c$ *and* the budget exception of Theorem 7.2 does not occur.

**Remark 13.2** (What this conjecture says, precisely)**.** Three details of the statement matter, and each of them is forced by Theorem 7.2.

*(a) The two clauses are conjoined, not disjuncted.* The classification is exact: the filter *fails* if and only if case (i) or case (ii) holds. Hence it *succeeds* if and only if case (i) fails (a non-residue is present) *and* case (ii) fails (the knapsack has a solution). A disjunction would assert the conjecture for primes for which every filter fails in both ways—that is, for the primes we are trying to find.

*(b) The condition is on $q = p(p+c)/4$, not on $x = (p+c)/4$ alone.* The criterion of Theorem 2.1 is a statement about divisors of $q^2$, and the obstruction of Theorem 7.1 is a statement about the prime factors of $q$. Since $q = p\,x$, the prime $p$ itself is a factor of $q$: whenever $\left(\frac{p}{c}\right) = -1$, the prime $p$ already supplies the non-residue, whatever the factorisation of $x$ looks like. Stating the condition on $x$ alone would silently discard that case (and would make the conjecture stronger than the theorem it rests on).

*(c) The bound is* 227*, not* 199*.* Theorem 10.7 exhibits a prime, 32 349 601, which no prime filter below 200 resolves; its prime-filter $c_{\min}$ is 227. With the bound 199 the conjecture as stated

is *false*. With the bound 227 it is true for every prime below $10^8$ that we have tested, and it is sharp: the survivor needs 227 among prime filters, and 15 among all filters (Remark 10.2).

**Remark 13.3.** Conjecture 13.1 is *sufficient* for the Erdős–Straus conjecture, and we have verified it for every prime $p \equiv 1 \pmod{12}$ below $10^7$ (Theorem 10.3). It is not known to be necessary: the conjecture might be provable for some $p$ by a first denominator outside the Bradford window, or by a Type-II split. We therefore present it as a sufficient reformulation that isolates the obstruction, not as an equivalent restatement.

The reformulation is nonetheless genuinely useful for two reasons. First, it converts the problem into a sieve question about linear forms, which is the setting in which the classical machinery (Mordell, Schinzel, Salez) operates and where the covering extension of Section 9 plugs in. Second, it makes the residual explicit: what remains is the set of primes $p$ for which $(p+c)/4$ is $c$-QR-pure for every $c \equiv 3 \pmod 4$ of sufficiently small modulus. Quantifying that set—rather than the $c = 3$-hard set—is the natural next question, and we expect it to be enormously rarer than the 21.5% of primes that fail only at $c = 3$.

## 14 Corrections to the monograph

**D1 — "density $\frac{1}{2}$" is an index, not a frequency**

The label on Theorem 4.1(a) of the monograph and the phrase "density-1/2 obstruction" in its Theorem 5.1(1) will be read as a failure rate. The measured rate is 3.54% (QR) and 1.42% (budget) at $c = 7$, i.e. a total $c = 7$ failure rate of 4.96%. The quantity 1/2 is the index of the QR subgroup in $(\mathbb{Z}/7\mathbb{Z})^\times$. See Table 2 for the measured densities of all filters.

**D2 — Listing 1 returns $(c, u)$ that contradict Theorem 1.2**

The algorithm of the monograph documents itself as returning $(c, u, x, y, z)$ with $c$ the modulus $d = 4x - n$ and $u$ the divisor of $q^2$ satisfying $u \equiv -q \pmod d$. In its even-$n$ branch it returns $(c, u) = (0, k)$ for $n = 2k$, where the criterion requires $d = n = 2k$ and $u = d$; in its $n \equiv 3 \pmod 4$ branch it returns $(c, u) = (1, a)$, where $d = 4a + 1$ and $u = 4a$. The returned *triples* are correct decompositions in both cases; only the metadata is wrong. Appendix A gives a corrected version in which the invariant $u \mid q^2$, $u \equiv v \equiv -q \pmod d$ is maintained in every branch.

**D3 — Corollary 2.3's "unit divisor $u = 1$"**

See Remark 4.5: $u = 1$ cannot satisfy criterion (2.1) because $-q \equiv 2 \pmod 3$ whereas $u = 1$. The identity is correct; the parenthetical justification is not.

**D4 — a false congruence in Table 2**

The cell of the monograph's Table 2 for $n = 1201$, $c = 15$ reads "$19 \equiv 1201 \equiv 1 \pmod{15}$". But $19 \equiv 4 \pmod{15}$. The *conclusion* of the cell is nevertheless correct: $D(15, q) = \{1, 2, 4, 8\}$ and $-q \equiv 11 \pmod{15}$, because $19 \equiv 4 = 2^2 \pmod{15}$ and $1201 \equiv 1 \pmod{15}$ both lie in the cyclic group $\langle 2 \rangle$ generated by 2 modulo 15. Only the stated residue is wrong.

Table 6: Literature audit. Every row was checked against primary sources (arXiv, Cambridge Core, Crossref, MathWorld).

| **As stated** | **As it is** |
|---|---|
| Swett (1999), *"The Erdős–Straus Conjecture: Computational Verification to* $10^{14}$*"*, Univ. of Indianapolis | Bound $10^{14}$ correct, but no such title exists: the work is a *web page*, *"The Erdős–Straus Conjecture"*, rev. 10/28/99. Crossref returns *zero* journal articles by Swett on this topic. Salez records the cost as 150 hours. |
| Salez (2014): complete set of seven modular equations, three new; $10^{17}$ in $\sim 16$ hours | Confirmed verbatim from arXiv:1406.6307. Precision: the 16 hours were on an AMD Turion II dual-core, so it is not a modern-machine figure. |
| "correcting the retracted 140 000 Salez filters figure" | The number 140 000 appears nowhere in Salez (2014); it belongs to Mihnea & Bogdan (2025), who *used* his algorithm ("$\lvert\mathcal{S}\rvert = 140000$"). *No retraction or correction of Salez exists*: arXiv:1406.6307 has a single version, no erratum. The monograph's wording implies a retraction that never happened. |
| Mihnea & Dumitru (2025), arXiv:2509.00128, $N = 10^{18}$ | Confirmed. Submitted 29 Aug 2025, *announced* September 2025 (the 2509 prefix is the announcement month). Runtime "about 2 weeks". arXiv metadata and the PDF byline disagree on the second author's name order. |
| Mordell (1969), *Diophantine Equations*, v. 30, pp. 287–290 | Book details correct—and 1969 is *more* correct than the 1967 propagated by Salez and Wikipedia. But Salez disputes the attribution: "it is absolutely incorrect to speak of 'Mordell's theorem"'; the identities are due to Rosati, Yamamoto and Schinzel, *reported in* Mordell's book. |
| Webb (1970), Vaughan (1970): the exceptional set has density zero | Both citations confirmed. Vaughan is quantitatively stronger: $\#\{n \le N\} \le N\exp(-c(\log N)^{2/3})$. Webb's own quantitative bound was paywalled and could not be verified; later work (Li, Yang, Ahmadi–Bleicher, Elsholtz) shares the credit. |

**A fifth, benign imprecision**

The monograph's Corollary 2.4 assumes $n$ prime where no primality is needed (see Remark 4.4). This is recorded here for completeness; the monograph's own proof does not use it.

## 15 Corrections to the literature review

## 16 Algorithms

**Proposition 16.1** (Linear-time filter decision)**.** *For a prime filter* $c \equiv 3 \pmod 4$ *and* $n \equiv 1 \pmod 4$*, the decision "does the filter* $c$ *succeed?" can be made in* $O\big(c\sum_i e_i\big)$ *time and* $O(c)$ *memory, where* $q = \prod p_i^{e_i}$*, instead of enumerating all* $\tau(q^2)$ *divisors.*

*Proof.* Let $g$ be a primitive root modulo $c$ and write $p_i \equiv g^{k_i}$. Then $c \mid q^2$ splits as $\sum_i a_i k_i \equiv (c-1)/2 + \sum_i e_i k_i \pmod{c-1}$ with $0 \le a_i \le 2e_i$, a bounded knapsack on $c-1$ states, solvable by dynamic programming with back-pointers. Note the constant $(c-1)/2$: it equals 3 only when $c = 7$, which is why the monograph's form of the knapsack is $c$-specific. □

**Remark 16.2.** The algorithm in the monograph recomputes the factorisation of $q$ for every candidate $c$ and then sorts all divisors of $q^2$. Replacing that by Proposition 16.1 and by a single sieve for the $c = 3$ census (see Appendix A) reduced the cost of the census to $10^6$ from 36 seconds to 0.1 seconds.

### 16.1 Composite filters: the decision procedure has to change

Section 10 and Theorem 10.7 show that composite filters are not optional — the survivor $32\,349\,601$ is caught by $c = 15$ — while Sections 7 and 16 treat only prime moduli. This subsection closes that gap, and it also corrects a defect in the algorithm printed in Appendix A.

**The defect.** The bounded-exponent knapsack of Proposition 16.1 decides the filter by writing each prime factor $p_i$ of $q$ as a power of a primitive root $g$ modulo $c$ and solving a linear congruence in the exponents modulo $c-1$. That is exact for prime $c$. For composite $c$ it is not merely inapplicable, it is wrong: the unit group $(\mathbb{Z}/c\mathbb{Z})^\times$ is not cyclic when $c$ is composite (for $c = 15$ it is $C_2 \times C_4$), so no primitive root exists; and if a prime $\pi \mid c$ also divides $q$, then $p_i \equiv 0 \pmod{\pi}$ has no discrete logarithm at all. An implementation that scans all $c \equiv 3 \pmod 4$ — as the monograph's own Listing 1 does — and then applies the prime-modulus knapsack to each of them is solving a different problem for the composite members of that list.

**The fix.** The decision "does there exist $u \mid q^2$ with $u \equiv -q \pmod c$?" can be made by a subset-product dynamic programme over the residues modulo $c$, with no primitive root and no discrete logarithm: for each prime power $p_i^{e_i} \parallel q$ the reachable set of residues of $\prod p_i^{a_i}$ $(0 \le a_i \le 2e_i)$ is updated, and the filter succeeds iff $-q$ is reachable. The cost is $O(c \sum e_i)$ time and $O(c)$ memory, and the witness is recovered by backtracking.

**Proposition 16.3** (The criterion is one congruence in the coprime case)**.** *Let $u, v, q, c$ be integers with $uv = q^2$, $\gcd(q, c) = 1$ and $c \mid u + q$. Then $c \mid v + q$. Consequently, when $\gcd(q, c) = 1$ the filter succeeds if and only if some divisor $u$ of $q^2$ satisfies $u \equiv -q \pmod c$.*

*Proof.* $c \mid q^2 + qv = q(q + v)$ and $\gcd(q, c) = 1$, so $c \mid q + v$ by Euclid's lemma. □

When $\gcd(q, c) > 1$ the second condition is genuinely independent of the first, because $u$ is not invertible modulo $c$: two divisors of $q^2$ that are congruent modulo $c$ can have different residues for $q^2/u$. In that case the implementation enumerates the divisors of $q^2$ instead. This is the branch that the printed listing of Appendix A now makes explicit.

**Validation.** The dynamic programme and the brute-force enumeration over all divisors of $q^2$ were compared on a large random sample of $(n, c)$ pairs with $n \equiv 1 \pmod 4$, including the composite moduli $c = 15, 27, 35, 39, 51, 55, 63, 75, 87, 91, 95, 99$ and the prime moduli $c = 3, 7, 11, \ldots, 199$. They agree on every case tested (`esfilter.py selftest`); the sample sizes and the number of mismatches are recorded in the accompanying data files.

**Remark 16.4** (What is true for composite $c$)**.** Three statements of this paper are statements about *prime* filters only, and we say so where they are used: the QR-pure obstruction (Theorem 7.1) and the exact failure classification (Theorem 7.2) are stated for prime $c$ because they are stated in terms of the Legendre symbol; the index-2 dichotomy (Corollary 7.3) is the same. For composite $c$ the corresponding obstruction is a statement about the characters of $(\mathbb{Z}/c\mathbb{Z})^\times$: a composite $c$ with $\omega(c)$ distinct odd prime factors has $[(\mathbb{Z}/c\mathbb{Z})^\times : (\mathbb{Z}/c\mathbb{Z})^{\times 2}] = 2^{\omega(c)}$, so there are *several* independent quadratic characters, and a divisor $u$ that is a square in one of them need not be a square in the others. This is exactly why $c = 15$ can succeed where $c = 3$ fails: the Legendre symbol $(\cdot/3)$ is not the only obstruction at $c = 15$, and the character $(\cdot/3)(\cdot/5)$ is a weaker one. We do not attempt a full classification for composite moduli here; the decision procedure of this subsection is exact for them, and that is what the census of Section 10 needs.

## 17 Verification methodology

Every assertion in this edition was checked by machine, from an implementation written from scratch (no third-party libraries), and independently by a second implementation built on a different strategy: solving $\frac{d}{q} = \frac{1}{y} + \frac{1}{z}$ by scanning $y$ directly, never touching the divisor criterion.

| **Check** | **Coverage** | **Result** |
|---|---|---|
| Criterion equivalence, both directions | 1 949 $(n, x)$ pairs, exhaustive $n \leq 120$ | 0 mismatches |
| Criterion vs. brute force | 500 random trials | 0 failures |
| Coprimality automaticity | 500 random trials | 0 failures |
| Trivial divisibility clause | 10 682 instances | 0 failures |
| mod4 barrier | 2 988 values of $n$, all $x$ | 0 violations |
| $k = 2$ escape refutation | 89 460 cases | 0 violations |
| Closed-form families | every $n \leq 5\,000$ | exact |
| $c = 3$ census | recomputed from scratch | matches exactly |
| Table 1 (nine decompositions) | exact rationals | all verified |
| Table 2 (18 diagnostic rows) | recomputed | all reproduced |
| Three regimes at $c = 7$ | 49 999 values of $n$ | 0 mismatches |
| Regime (c) sub-case | every instance of the shape | exact |
| Thms. 7.1, 7.2 | 225 365 failing $(n, c)$ pairs | 0 violations |
| Theorem 8.1 | 807 instances | 0 violations |
| Covering identities (Table 3) | 15 780 identities | 0 violations |
| $c_{\min}$ census | 166 011 primes | 0 survivors |

## 18 Conclusion and recommendations

The audited monograph is mathematically sound and deserves to be cited for its verification layer; it should not be cited as a new result. This edition adds three things it does not have: an *exact* failure criterion for prime filters (Theorem 7.2), a one-line identity covering every filter at once (Theorem 8.1), and a computation that replaces a table of special cases with a landscape (Section 10).

Our practical recommendations for anyone building on this machinery are:

1. Decide filters with the knapsack (Proposition 16.1), not by enumerating divisors of $q^2$. The target constant is $(c-1)/2$, which coincides with the monograph's 3 only at $c = 7$.

2. Expect to walk to $c \approx 200$. The monograph's tables stop at $c = 23$; below $10^8$ there is a prime requiring $c = 227$ (Theorem 10.7).

3. Include composite filters. The single survivor below $10^8$ is invisible to every prime filter below 200 and is caught at $c = 15$.

4. Measure a covering as a union, not as a sum. The classical covering of the six Mordell classes reaches 38%–43%, not the 58%–61% that summing $1/m$ over overlapping classes reports (Remark 9.3).

5. Use the whole parametric family. The 27 divisors $\ell^r n^a w^b$ of $q^2$ take the covered fraction from 43% to 97.6%–97.9% (Section 9.1).

6. Track $c_{\min}$, not the density of failures. The density of failures decays like a power of $N$, and its tail is what determines the engineering cost. Over all primes below $10^8$ the monograph's own algorithm never needs $c > 107$.

7. Do not expect help from a moving modulus. The Type II family (Theorem 12.2) never improves on the Type I family; the single-term framework is exhausted.

8. Formalise as you go. The proofs in this edition are checked by the Lean kernel (Section 19); the finite statements—the census, the published decompositions, the survivor—are machine-checked theorems, not computations we merely report.

The conjecture itself remains open. What this edition does is to convert it, for the prime case, into a single sieve-theoretic question about the prime factorisations of the linear shifts $(p+3)/4, (p+7)/4, \ldots$ (Conjecture 13.1), to verify that question on 1.44 million primes, and to exhibit—in the single number 32 349 601—a concrete instance where the classical obstructions and the prime-filter mechanism both fail before yielding to a composite filter.

## 19 Formal verification in Lean 4

The core of this edition has been formalised in Lean 4 (theorems proved, not merely checked): the criterion, the barrier, the generalized identity, the unconditional families and the finite census. The development is small and self-contained; it uses Mathlib only for `ring`, `nlinarith`, `linear_combination`, `omega` and `native_decide`. The source files accompany this document and are reproduced below, verbatim.

### 19.1 What is formalised

| **Statement in this document** | **Lean file and theorem** |
|---|---|
| Theorem 2.1 (the criterion, both directions) | `Criterion.lean`: `es_iff_criterion`, `criterion_iff_es`, `criterion_witness` |
| The same criterion with positivity (the Egyptian-fraction statement) | `Criterion.lean`: `es_iff_criterion_pos`, `criterion_pos_witness_pos` |
| Theorem 3.1 (the mod4 barrier) | `Barrier.lean`: `barrier`, `barrier_modEq` |
| Proposition 3.2 (the $k = 2$ escape fallacy) | `Barrier.lean`: `k2_escape_fallacy`, `k2_modulus`, `modulus_odd` |
| Theorem 8.1 (generalized filter identity) | `Identity.lean`: `generalized_filter_identity` (and its `_rat` version) |
| Theorem 8.4 (the Dual Filter Identity), and Proposition 16.3 (the criterion is one congruence) | `Identity.lean`: `split_identity_rat`, `dual_filter_divisibility`, `criterion_reduced_nat` |
| Theorem 4.1, Corollary 4.2, Corollary 4.3 | `Families.lean`: `family_even`, `family_three_mod_four`, `family_three_mod_four_distinct`, `corollary_2_3`, `corollary_2_4` |
| Theorem 5.2 (the nine $c = 3$-hard primes below 1000) | `Finite.lean`: `c3_hard_below_1000` |
| Table 1 (nine $c = 7$ decompositions) and Table 2 | `Finite.lean`: `table1_73` …, `table2_29` … |
| Theorem 10.7 (the survivor below $10^8$ and its two decompositions; *not* its uniqueness—see the scope remark) | `Finite.lean`: `survivor_32349601`, `survivor_32349601_at_227`, `survivor_no_small_prime_filter`, `survivor_class` |
| Mordell's six uncovered classes | `Finite.lean`: `mordell_uncovered_classes` |

**Remark 19.1.** The census and the survivor statements are proved by `native_decide`: the predicate is compiled to native code and the result is checked by the Lean kernel. These are therefore machine-checked finite theorems, not merely computations that we report.

**Remark 19.2** (No axioms beyond the Lean standard three)**.** Every theorem listed above is closed under `#print axioms`: each depends only on `propext`, `Classical.choice`

and `Quot.sound` (and, for the `native_decide` statements, on `Lean.ofReduceBool`). In particular *no theorem depends on* `sorryAx`: there is no gap, no admitted step and no metamathematical assumption anywhere in the development.

**Remark 19.3** (Scope)**.** Three statements of this document are *not* formalised, and we state them here rather than leave them implicit.

1. The exact failure classification of Theorem 7.2 (it needs a formal account of prime factorisations inside a residue-class argument).
2. The covering extension of Theorem 9.1 (a CRT statement over many moduli).
3. The *uniqueness* of the survivor in Theorem 10.7. What Lean proves is that $32\,349\,601$ has no prime filter below 200 and that it has the two displayed decompositions; the statement that it is the *only* such prime below $10^8$ is a finite computation over $1\,440\,021$ primes (certified by the script and the data files that accompany this document), not a Lean theorem.

All three are proved in the text and verified computationally.

## 19.2 The development

```
import Mathlib.Data.Nat.Factorization.Basic
import Mathlib.Data.Int.GCD
import Mathlib.Data.Nat.ModEq
import Mathlib.Algebra.BigOperators.Group.Finset.Basic
import Mathlib.Tactic.NormNum
import Mathlib.Tactic.Ring
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.LinearCombination
import Mathlib.Tactic.FieldSimp
import Lean

/-!
# Single-term extraction for the Erdos-Straus equation

Core definitions shared by the whole development.

The Erdos-Straus equation is `4/n = 1/x + 1/y + 1/z`.  Clearing denominators
(this is valid over `Int`, and for positive `x y z` equivalent to the rational
equation) gives the `ES` predicate below.

After extracting a first denominator `x`, the remainder is

    4/n - 1/x = (4x - n) / (n x) = d / q,

and the monograph's divisor criterion says that `d/q` splits into two unit
fractions `1/y + 1/z` if and only if there is a divisor `u | q^2` with
`u = -q (mod d)` and `q^2/u = -q (mod d)`.  We state the criterion in the
equivalent, division-free form: there are `u v` with `u * v = q^2`,
`d | (q + u)` and `d | (q + v)`.
-/

namespace ErdosStraus
```

Listing 1: `ErdosStraus/Basic.lean` — definitions (part 1 of 2).

```
33
34 /-- The Erdos-Straus equation `4/n = 1/x + 1/y + 1/z`, cleared of denominators.
       -/
35 def ES (n x y z : Int) : Prop := 4 * x * y * z = n * (y * z + x * z + x * y)
36
37 /-- The Erdos-Straus equation with the positivity hypotheses that make it a
38 statement about Egyptian fractions rather than about arbitrary integers.  The
39 predicate `ES` alone allows zero and negative denominators. -/
40 def ESpos (n x y z : Int) : Prop :=
41   0 < n ∧ 0 < x ∧ 0 < y ∧ 0 < z ∧ 4 * x * y * z = n * (y * z + x * z + x * y)
42
43 /-- Numerator of the remainder `4/n - 1/x`. -/
44 def remNum (n x : Int) : Int := 4 * x - n
45
46 /-- Denominator of the remainder `4/n - 1/x`. -/
47 def remDen (n x : Int) : Int := n * x
48
49 /-- The single-term-extraction criterion (*) of the monograph, in division-free
50 form: the two-term split of `d/q` exists if and only if `q^2` factors as
51 `u * v` with `u = v = -q (mod d)`. -/
52 def Criterion (n x : Int) : Prop :=∃
53    u v : Int, u * v = remDen n x ^ 2 ∧
54     remNum n x | (remDen n x + u) ∧ remNum n x | (remDen n x + v)
55
56 /-- The rational form of the Erdos-Straus equation. -/
57 def ESrat (n x y z : Rat) : Prop := (4 / n : Rat) = 1 / x + 1 / y + 1 / z
58
59 /-- Cancellation of a nonzero factor on the left (ASCII-named wrapper). -/
60 theorem mul_cancel_left {a b c : Int} (ha : a ≠ 0) (h : a * b = a * c) : b = c
       :=₀
61   mul_left_cancel ha h
62
63 /-- The first denominator produced by the criterion. -/
```

Listing 2: `ErdosStraus/Basic.lean` — definitions (part 2 of 2).

```
import ErdosStraus.Basic

/-!
# The unified single-term-extraction criterion

Theorem 2.1 of the corrected edition, formalised.  The two directions are proved
separately; the `Int` setting keeps all the algebra inside a ring, which is
what makes the arithmetic steps (`linear_combination`, `nlinarith`) go through
without any truncated-subtraction case analysis.
-/

namespace ErdosStraus

theorem es_iff_criterion {n x y z : Int} (hd : 0 < remNum n x) (h : ES n x y z)
    :
    Criterion n x := by
  -- the key identity: (4x - n) y z = n x (y + z)
  have key : (4 * x - n) * y * z = n * x * (y + z) := by
    have h1 : 4 * x * y * z = n * (y * z + x * z + x * y) := h
    nlinarith [h1]
  have key' : remNum n x * y * z = remDen n x * (y + z) := by
    simpa [remNum, remDen] using key
  refine ⟨remNum n x * y - remDen n x, remNum n x * z - remDen n x, ?_, ?_, ?⟩_
  . -- the two factors multiply to q^2
    have huv : (remNum n x * y - remDen n x) * (remNum n x * z - remDen n x)
        = remDen n x ^ 2 := by
      simp only [remNum, remDen]
      linear_combination (4 * x - n) * key
    exact huv
  . -- d | (q + u), because q + u = d * y
    have hq : remDen n x + (remNum n x * y - remDen n x) = remNum n x * y := by
        ring
    rw [hq]
    exact Int.dvd_mul_right _ _
  . -- d | (q + v), because q + v = d * z
    have hq : remDen n x + (remNum n x * z - remDen n x) = remNum n x * z := by
        ring
    rw [hq]
    exact Int.dvd_mul_right _ _

theorem criterion_iff_es {n x : Int} (hd : 0 < remNum n x) :
    Criterion n x ↔∃  y z : Int, ES n x y z := by
  constructor
  . rintro ⟨u, v, huv, hu, ⟩hv
    rcases hu with ⟨k, ⟩hk     -- q + u = d * k
    rcases hv with ⟨l, ⟩hl
    refine ⟨k, l, ?⟩_
    have hk' : remNum n x * k = remDen n x + u := by
      have h := hk; omega
    have hl' : remNum n x * l = remDen n x + v := by
      have h := hl; omega
```

Listing 3: `ErdosStraus/Criterion.lean` — Theorem 1.2, both directions (part 1 of 2).

```
⟨⟩↔∃⟨⟩⟨⟩⟨⟩⟨⟩
    -- (d k - q)(d l - q) = u v = q^2
    have hfac : (remNum n x * k - remDen n x) * (remNum n x * l - remDen n x)
        = remDen n x ^ 2 := by
      rw [hk', hl']
      linear_combination huv
    -- cancel one factor d to get d k l = q (k + l)
    have hkey : remNum n x * (remNum n x * (k * l))
        = remNum n x * (remDen n x * (k + l)) := by
      linear_combination hfac
    have hd0 : remNum n x ≠ 0 := by
      have h := hd; omega
    have hkey' : remNum n x * (k * l) = remDen n x * (k + l) :=
      mul_cancel_left hd0 hkey
    -- convert d k l = q (k+l) into the Erdos-Straus equation
    have hk1 : (4 * x - n) * (k * l) = n * x * (k + l) := by
      simpa [remNum, remDen] using hkey'
    simp only [ES]
    linear_combination hk1
  . rintro ⟨y, z, ⟩h
    exact es_iff_criterion hd h

/-- The denominators recovered from the criterion are exactly `(q+u)/d` and
`(q+v)/d`. -/
theorem criterion_witness {n x u v k l : Int}
    (hd : remNum n x ≠ 0)
    (hk : remNum n x * k = remDen n x + u)
    (hl : remNum n x * l = remDen n x + v)
    (huv : u * v = remDen n x ^ 2) :
    ES n x k l := by
  have hfac : (remNum n x * k - remDen n x) * (remNum n x * l - remDen n x)
      = remDen n x ^ 2 := by
    rw [hk, hl]
    linear_combination huv
  have hkey : remNum n x * (remNum n x * (k * l))
      = remNum n x * (remDen n x * (k + l)) := by
    linear_combination hfac
  have hd0 : remNum n x ≠ 0 := by
    have h := hd
    omega
  have hkey' : remNum n x * (k * l) = remDen n x * (k + l) :=
    mul_cancel_left hd0 hkey
  have hk1 : (4 * x - n) * (k * l) = n * x * (k + l) := by
    simpa [remNum, remDen] using hkey'
  simp only [ES]
  linear_combination hk1

/-- **The criterion for positive denominators.**  `ESpos` is the honest
```

Listing 4: `ErdosStraus/Criterion.lean` — Theorem 1.2, both directions (part 2 of 2).

```
import ErdosStraus.Basic

/-!
# The `mod 4` barrier and the refutation of the `k = 2` escape

Theorem 3.1 of the corrected edition, and the refutation of the heuristic that tries to
escape the quadratic-residue obstruction by taking `x = (n + c) / 2`.
-/

namespace ErdosStraus

/-- **Theorem 3.1 of the corrected edition (universal barrier).**  For every `n = 1 (mod 4)` and every
first denominator `x`, the modulus `d = 4x - n` is `3 (mod 4)`. -/
theorem barrier {n x : Nat} (hn : n % 4 = 1) (hx : 4 * x - n > 0) :
    (4 * x - n) % 4 = 3 := by omega

/-- The same statement for the modulus `d` in `Int` form: `d == 3 (mod 4)`. -/
theorem barrier_modEq {n x : Int} (hn : n % 4 = 1) (hx : 0 < 4 * x - n) :
    (4 * x - n) % 4 = 3 := by omega

/-- **Refutation of the `k = 2` escape.**  With `x = (n + c) / 2` and `c` odd,
the modulus is `d = n + 2c`, which is still `3 (mod 4)`; the shift `c` is never the modulus. -/
theorem k2_escape_fallacy {n c : Nat} (hn : n % 4 = 1) (hc : c % 2 = 1) :
    (n + 2 * c) % 4 = 3 := by omega

/-- The modulus of the `k = 2` escape really is `n + 2c`. -/
theorem k2_modulus {n c x : Nat} (h : 2 * x = n + c) :
    4 * x - n = n + 2 * c := by omega

/-- For odd `n` the modulus `d = 4x - n` is always odd, so no factor of `2`
can cancel between `d` and `q = n x`. -/
theorem modulus_odd {n x : Nat} (hn : n % 2 = 1) (hx : n ≤ 4 * x) :
    (4 * x - n) % 2 = 1 := by omega

end ErdosStraus
```

Listing 5: `ErdosStraus/Barrier.lean` — the mod4 barrier and the $k = 2$ refutation.

```
import ErdosStraus.Basic

/-!
# The generalized filter identity

The Generalized Filter Identity (new in this edition, Theorem 8.1 of the
corrected edition) states that for *any*
filter `c` and any prime `l` dividing `x = (n + c)/4` with `c | (n + l)`,

    4/n = 1/x + 1/(x (n + l)/c) + 1/(x n (n + l)/(c l)).

It is one line of algebra, it subsumes the monograph's Corollary 2.4, and it
drops that corollary's unnecessary primality hypothesis.
-/

namespace ErdosStraus

/-- The cleared-denominator form follows from the rational one. -/
theorem ES_of_ESrat {n x y z : Int} (hn : n ≠ 0) (hx : x ≠ 0) (hy : y ≠ 0) (hz :
     z ≠ 0)
    (h : ESrat n x y z) : ES n x y z := by
  have h1 := h
  simp only [ESrat] at h1
  field_simp at h1
  ring_nf at h1
  have h' : 4 * x * y * z = n * (y * z + x * z + x * y) := by
    have h2 : (4 : Rat) * x * y * z = (n : Rat) *
        ((y : Rat) * (z : Rat) + (x : Rat) * (z : Rat) + (x : Rat) * (y : Rat))
            := by
      have h3 := h1
      convert h3 using 1 <;> ring
    exact_mod_cast h2
  exact h'

/-- **Generalized Filter Identity**, in rational form.

The hypotheses `hxc` and `hnl` record that the denominators do not vanish;
they are the arithmetic content of `x ≠ 0` and `n + l ≠ 0`. -/
theorem generalized_filter_identity_rat {n c x l y z : Rat}
    (hx : 4 * x = n + c)
    (hly : c * y = x * (n + l))
    (hlz : c * l * z = x * n * (n + l))
```

Listing 6: `ErdosStraus/Identity.lean` — the Generalized Filter Identity (part 1 of 2).

```
======
    (hc : c ≠ 0) (hl : l ≠ 0) (hn : n ≠ 0) (hxc : n + c ≠ 0) (hnl : n + l ≠ 0) :
    ESrat n x y z := by
  simp only [ESrat]
  have hy' : y = x * (n + l) / c := by
    field_simp
    linarith [hly]
  have hz' : z = x * n * (n + l) / (c * l) := by
    field_simp
    linarith [hlz]
  rw [hy', hz']
  have hx' : x = (n + c) / 4 := by
    field_simp
    linarith [hx]
  rw [hx']
  field_simp [hc, hl, hn, hxc, hnl]
  ring

/-- **Generalized Filter Identity**, in cleared-denominator form over `Int`. -/
theorem generalized_filter_identity {n c x l y z : Int}
    (hx : 4 * x = n + c)
    (hly : c * y = x * (n + l))
    (hlz : c * l * z = x * n * (n + l))
    (hc0 : c ≠ 0) (hl0 : l ≠ 0) (hn0 : n ≠ 0) (hx0 : x ≠ 0) (hy0 : y ≠ 0) (hz0 :
        z ≠ 0)
    (hnl0 : n + l ≠ 0) (hxc0 : n + c ≠ 0) :
    ES n x y z := by
  have hr : ESrat (n : Rat) (x : Rat) (y : Rat) (z : Rat) := by
    refine generalized_filter_identity_rat (n := (n : Rat)) (c := (c : Rat)) (x
        := (x : Rat))
      (l := (l : Rat)) (y := (y : Rat)) (z := (z : Rat)) ?_ ?_ ?_ ?_ ?_ ?_ ?_ ?_
    . exact_mod_cast hx
    . exact_mod_cast hly
    . exact_mod_cast hlz
    . exact_mod_cast hc0
    . exact_mod_cast hl0
    . exact_mod_cast hn0
    . exact_mod_cast hxc0
    . exact_mod_cast hnl0
  exact ES_of_ESrat hn0 hx0 hy0 hz0 hr
```

Listing 7: `ErdosStraus/Identity.lean` — the Generalized Filter Identity (part 2 of 2).

```
import ErdosStraus.Basic

/-!
# The three unconditional families

Theorem 4.1 and Corollaries 4.2--4.3 of the corrected edition, in rational form so
that the divisions are handled by `field_simp` rather than by case analysis on
truncated subtraction.
-/

namespace ErdosStraus

/-- The rational Erdos-Straus equation on natural-number denominators. -/
def ESratN (n x y z : Nat) : Prop :=
  (4 / n : Rat) = (1 / x : Rat) + (1 / y : Rat) + (1 / z : Rat)

/-- **Family 1 (even `n`).**  `4/(2k) = 1/k + 1/(k+1) + 1/(k(k+1))`. -/
theorem family_even (k : Nat) (hk : 0 < k) :
    ESratN (2 * k) k (k + 1) (k * (k + 1)) := by
  have hk0 : (k : Rat) ≠ 0 := by exact_mod_cast (Nat.ne_of_gt hk)
  simp only [ESratN]
  field_simp [hk0]
  ring

/-- **Family 2 (`n = 3 (mod 4)`).**  With `n = 4a - 1`,
`4/n = 1/(2a) + 1/(2a) + 1/(an)`. -/
theorem family_three_mod_four (n a : Rat) (h : 4 * a = n + 1) (hn : n ≠ 0)
    (hn1 : n + 1 ≠ 0) :
    ESrat n (2 * a) (2 * a) (a * n) := by
  simp only [ESrat]
  have ha' : a = (n + 1) / 4 := by
    field_simp
    linarith [h]
  rw [ha']
  field_simp [hn, hn1]
  ring

/-- **Family 2', distinct denominators.**  With `n = 4a - 1`,
```

Listing 8: `ErdosStraus/Families.lean` — the unconditional families (part 1 of 2).

```
≠≠≠
`4/n = 1/a + 1/(an+1) + 1/(an(an+1))`, in cleared-denominator form. -/
theorem family_three_mod_four_distinct {n a : Int} (h : 4 * a = n + 1) :
    ES n a (a * n + 1) (a * n * (a * n + 1)) := by
  have h1 : 4 * a * (a * n + 1) * (a * n * (a * n + 1))
      = n * ((a * n + 1) * (a * n * (a * n + 1)) + a * (a * n * (a * n + 1))
        + a * (a * n + 1)) := by
    linear_combination (a * n * (a * n + 1) ^ 2) * h
  exact h1

/-- **Corollary 2.3 (`n = 5 (mod 12)`).**  With `a = (n+3)/4`,
`4/n = 1/a + 1/(a(n+1)/3) + 1/(an(n+1)/3)`. -/
theorem corollary_2_3 (n a : Rat) (h : 4 * a = n + 3) (hn : n ≠ 0) (hn3 : n + 3
    ≠ 0)
    (hn1 : n + 1 ≠ 0) :
    ESrat n a (a * (n + 1) / 3) (a * n * (n + 1) / 3) := by
  simp only [ESrat]
  have ha' : a = (n + 3) / 4 := by
    field_simp
    linarith [h]
  rw [ha']
  field_simp [hn, hn3, hn1]
  ring

/-- **Corollary 2.4 (`n = 1 (mod 12)`, a divisor `l = 2 (mod 3)` of `a`).**
No primality hypothesis on `n` is needed---this is the sharpening noted in the
audit. -/
theorem corollary_2_4 (n a l : Rat) (h : 4 * a = n + 3) (hn : n ≠ 0) (hn3 : n +
    3 ≠ 0)
    (hnl : n + l ≠ 0) (hl : l ≠ 0) :
    ESrat n a (a * (n + l) / 3) (a * n * (n + l) / (3 * l)) := by
  simp only [ESrat]
  have ha' : a = (n + 3) / 4 := by
    field_simp
    linarith [h]
  rw [ha']
  field_simp [hn, hn3, hnl, hl]
  ring

end ErdosStraus
```

Listing 9: `ErdosStraus/Families.lean` — the unconditional families (part 2 of 2).

```
import ErdosStraus.Basic

/-!
# Machine-checked finite statements

Everything in this file is decided by `native_decide`, i.e. it is checked by
the Lean kernel *after* the code has been compiled to native code.  These are
the census of `c = 3`-hard primes below 1000, the published `c = 7`
decompositions, and the decomposition of the unique prime below `10^8` that
resists every prime filter `c <= 199`.
-/

namespace ErdosStraus

/-- Increment the multiplicity of the prime `p` in a factor list. -/
def incPrime : Nat -> List (Prod Nat Nat) -> List (Prod Nat Nat)
  | p, [] => [(p, 1)]
  | p, (q, e) :: rest => if q = p then (q, e + 1) :: rest else (q, e) ::
      incPrime p rest

/-- Trial-division factorisation, with `fuel` guaranteeing termination. -/
def factorAux : Nat -> Nat -> Nat -> List (Prod Nat Nat) -> List (Prod Nat Nat)
  | 0, n, _, acc => if n > 1 then (n, 1) :: acc else acc
  | fuel + 1, n, d, acc =>
      if d * d > n then (if n > 1 then (n, 1) :: acc else acc)
      else if n % d == 0 then factorAux fuel (n / d) d (incPrime d acc)
      else factorAux fuel n (d + 1) acc

/-- The prime factorisation of `n` as a list of (prime, multiplicity) pairs. -/
def factorPairs (n : Nat) : List (Prod Nat Nat) := factorAux (n + 1) n 2 []

/-- Primality by trial division; decidable and `native_decide`-friendly.

`factorPairs` already stops as soon as `d * d > n`, so testing primality this
way costs `O(sqrt n)` rather than the `O(n)` of a naive trial division. -/
def isPrimeB (n : Nat) : Bool :=
  n >= 2 && factorPairs n == [(n, 1)]

/-- Every divisor of `n^2`, computed from the prime factorisation of `n`. -/
def divisorsOfSquare (n : Nat) : List Nat :=
  (factorPairs n).foldl
    (fun acc (p, e) =>
      acc.flatMap (fun d => (List.range (2 * e + 1)).map (fun a => d * p ^ a)))
    [1]

/-- Does the filter `c` succeed for `p`?  Criterion (*) with `x = (p+c)/4`,
`d = c`, `q = p*x`. -/
def filterWorks (p c : Nat) : Bool :=
  if (p + c) % 4 != 0 then false
  else
    let x := (p + c) / 4
    let q := p * x
    (divisorsOfSquare q).any (fun u =>
```

Listing 10: `ErdosStraus/Finite.lean` — the machine-checked finite statements (part 1 of 2).

```
      (q + u) % c == 0 && (q + q * q / u) % c == 0)

/-- **The census of Theorem 5.2**: exactly nine primes below 1000 are
`c = 3`-hard, and they are precisely those displayed. -/
theorem c3_hard_below_1000 :
    (List.filter (fun p => isPrimeB p && p % 12 == 1 && !filterWorks p 3)
      (List.range 1000))
      = [73, 193, 241, 313, 409, 433, 601, 673, 769] := by native_decide

/-- The nine published `c = 7` decompositions (Table 1 of the monograph). -/
theorem table1_73  : ES 73 20 210 30660 := by norm_num [ES]
theorem table1_193 : ES 193 50 1380 1331700 := by norm_num [ES]
theorem table1_241 : ES 241 62 2139 1030998 := by norm_num [ES]
theorem table1_313 : ES 313 80 3580 4482160 := by norm_num [ES]
theorem table1_409 : ES 409 104 6135 638040 := by norm_num [ES]
theorem table1_433 : ES 433 110 6805 64824430 := by norm_num [ES]
theorem table1_601 : ES 601 152 13053 62758824 := by norm_num [ES]
theorem table1_673 : ES 673 170 16345 374006290 := by norm_num [ES]
theorem table1_769 : ES 769 194 21340 16410460 := by norm_num [ES]

/-- The published decompositions for the regression suite (Table 2). -/
theorem table2_29    : ES 29 8 80 2320 := by norm_num [ES]
theorem table2_1129  : ES 1129 285 29354 8365890 := by norm_num [ES]
theorem table2_1201  : ES 1201 306 15980 172727820 := by norm_num [ES]
theorem table2_2521  : ES 2521 636 70588 5611746 := by norm_num [ES]

/-- The unique prime below `10^8` that resists every prime filter `c <= 199`
is `32349601`, and it is caught by the composite filter `c = 15`. -/
theorem survivor_32349601 :
    ES 32349601 8087404 17441619559583 5257444201599103534852 := by norm_num [ES
        ]

/-- ... and by the prime filter `c = 227`. -/
theorem survivor_32349601_at_227 :
    ES 32349601 8087457 3144834111614 1819276861038 := by norm_num [ES]

/-- No prime filter `c <= 199` works for `32349601`. -/
theorem survivor_no_small_prime_filter :
    (List.filter (fun c => c % 4 == 3 && isPrimeB c) (List.range 200)).all
      (fun c => filterWorks 32349601 c == false) = true := by native_decide

/-- The survivor lies in one of Mordell's six uncovered classes,
`361 = 19^2 (mod 840)`. -/
theorem survivor_class : 32349601 % 840 = 361 := by norm_num

/-- The six classes left uncovered by Mordell's identities are exactly the
squares of the units modulo 840. -/
theorem mordell_uncovered_classes :
    (List.range 840).filter (fun r =>
        (List.range 840).filter (fun u => Nat.gcd u 840 = 1) |>.map
          (fun u => u * u % 840) |>.contains r)
      = [1, 121, 169, 289, 361, 529] := by native_decide
```

Listing 11: `ErdosStraus/Finite.lean` — the machine-checked finite statements (part 2 of 2).

## References

[1] L. J. Mordell, *Diophantine Equations*, Pure and Applied Mathematics Vol. 30, Academic Press, London–New York, 1969, pp. 287–290.

[2] A. Schinzel, as reported in [1]; see also Salez's discussion of the attribution.

[3] W. A. Webb, "On $4/n = 1/x + 1/y + 1/z$", *Proc. Amer. Math. Soc.* **25** (1970), 578–584.

[4] R. C. Vaughan, "On a problem of Erdős, Straus and Schinzel", *Mathematika* **17** (1970), 193–198.

[5] A. Swett, "The Erdős–Straus Conjecture", *Current Research on ESC*, Univ. of Indianapolis, rev. 10/28/99.

[6] S. Salez, "The Erdős–Straus conjecture: New modular equations and checking up to $N = 10^{17}$", arXiv:1406.6307, 2014.

[7] S. Mihnea and B. C. Dumitru, "Further verification and empirical evidence for the Erdős–Straus conjecture", arXiv:2509.00128, 2025.

[8] `advance_results_1e7.json`, `verification_results.json` (this audit), and `es_solver.py`; see Appendix A.

## A Corrected algorithm

```
def split_remainder(n, x):
    """Theorem 2.1: split 4/n - 1/x = d/q into 1/y + 1/z.
    Returns (d, q, u, v, y, z) or None.  The returned tuple satisfies
    Theorem 2.1 exactly:  u | q^2,  u = -q (mod d),  v = q^2/u = -q (mod d)."""
    d, q = 4 * x - n, n * x
    if d <= 0 or q <= 0:
        return None
    if d == 1:
        return (d, q, 1, q * q, q + 1, q * q + q)
    fac = factorize(q)
    if q % d == 0:                      # Prop 2.3(2): trivial divisibility
        m = q // d
        return (d, q, d, q * q // d, m + 1, m * (m + 1))
    if is_prime(d):                     # prime modulus: bounded-exponent DP
        ok, wit = knapsack_witness(d, fac)      # O(d * sum e_i), no factoring
        if not ok:
            return None
        u = 1
        for p, a in wit.items():
            u *= p ** a
    else:
        # Composite modulus: (Z/dZ)* has no primitive root, so the
        # discrete-log DP does not apply.  Fall back to divisor enumeration.
        u = None
        for cand in divisors_of_square(fac):
            if (q + cand) % d == 0 and (q + q * q // cand) % d == 0:
                u = cand
                break
        if u is None:
            return None
    v = q * q // u
    assert (q + u) % d == 0 and (q + v) % d == 0
    return (d, q, u, v, (q + u) // d, (q + v) // d)
```

```

def solve(n):
    """Return (c, u, x, y, z) with 4/n = 1/x + 1/y + 1/z, or None.

    The metadata (c, u) is the modulus and the witness divisor of
    Theorem 2.1, in every branch."""
    if n % 2 == 0:                      # Theorem 5.2, case 1:  4/2k
        k = n // 2
        return (n, n, k, k + 1, k * (k + 1))
    if n % 4 == 3:                      # Theorem 5.2, case 2:  x = 2a
        a = (n + 1) // 4
        return (4 * a + 1, 4 * a, 2 * a, 2 * a, a * n)
    for c in range(3, n + 3, 4):        # filters c = 3 (mod 4), prime or
        composite
        if (n + c) % 4:
            continue
        r = split_remainder(n, (n + c) // 4)
        if r:
            d, q, u, v, y, z = r
            return (d, u, (n + c) // 4, y, z)
    return None                        # no filter inside the Bradford window
```

Listing 12: Corrected single-term extraction with the bookkeeping of Theorem 2.1 maintained in every branch, and with the prime/composite split of the decision procedure made explicit.

**Remark A.1.** The assert line is the invariant the monograph's Listing 1 fails to keep in its even-$n$ and $n \equiv 3 \pmod 4$ branches (defect D2): the returned tuple $(c, u, x, y, z)$ always satisfies $u \mid q^2$ and $u \equiv v \equiv -q \pmod c$ with $q = nx$ and $c = d = 4x - n$, in every branch—including the two branches that do not come from a filter at all, where the correct metadata is $(c, u) = (n, n)$ for $n = 2k$ and $(c, u) = (4a + 1, 4a)$ for $n = 4a - 1$.

The listing also makes explicit the prime/composite split that the monograph's algorithm hides: the bounded-exponent knapsack of Proposition 16.1 is a decision procedure over $\mathbb{Z}/(c-1)\mathbb{Z}$ and therefore requires a primitive root, which exists only for prime $c$. For composite $c$ the listing falls back to divisor enumeration. The complete implementation, with the sieve-based census and the command-line interface, accompanies this document as `es_solver.py`.

# B Reproducibility

```
python3 es_solver.py 1201        # 4/1201 = 1/306 + 1/1082101 + 1/16218
python3 es_solver.py --cmin 2521 # 23
python3 es_solver.py --classify 1201
python3 es_solver.py --census 1000000
python3 es_solver.py --scan 200000        # every n in [2, 200000] decomposes
python3 advance.py 10000000 advance_results_1e7.json
```

Listing 13: Reproducing the principal computations.

The census in Table 5 and Table 2 is produced by `advance.py`, which implements the decision procedure of Proposition 16.1 and verifies Theorem 8.1 and Theorem 7.2 as it goes.

## B.1 A certificate for the census

The statement that the algorithm resolves every prime below $10^8$ is a finite claim about 1 440 021 primes, and finite claims of that kind are worth more with a witness than with a promise. We therefore ship the witness: `certificate_1e7.jsonl` contains, for every prime $p \equiv 1$

(mod 12) below $10^7$ that the algorithm resolves, the record $(p, c, u, x, y, z)$ with $u \mid q^2$, $u \equiv -q \pmod{c}$, $q^2/u \equiv -q \pmod{c}$ and $4/p = 1/x + 1/y + 1/z$ exactly.

`check_certificate.py` re-verifies the file with its own arithmetic — it shares no code with the generator — and prints the SHA-256 of the file, so that the certificate can be checked against the copy that was actually used. The certificate shipped with this edition contains 166 011 records, one for each prime $p \equiv 1 \pmod{12}$ below $10^7$; all of them verify, none is left unresolved, and its SHA-256 is `dc0fff3b3004e8a506ecff9b69d164c40cad92faa19e7c10e3cf0d38f6b8`

```
python3 esfilter.py certificate 10000000 certificate_1e7.jsonl
python3 check_certificate.py certificate_1e7.jsonl --sha256 <hash>
```

Listing 14: Verifying the certificate.

We also ship the extended covering of Section 9 (`covering_ext.py`, `covering_ext_results.json`) and the composite-safe decision procedure of Section 16.1 (`esfilter.py`, together with its differential test against brute force).